

Problema de Programação: Sistema de Micromobilidade Urbana (Bike-Sharing)

Cenário de Contexto

A orientação a objetos nos permite modelar entidades do mundo real ou conceitos abstratos agrupando dados (atributos) e comportamentos (métodos) em unidades lógicas. Para este exercício, você deverá implementar o núcleo de simulação de um **Sistema de Compartilhamento de Bicicletas Elétricas**.

O sistema deve modelar a interação entre os usuários e as bicicletas disponíveis na cidade por meio da **troca de mensagens** entre os objetos.

Requisitos do Projeto

Você deverá criar **duas classes** em Python: **Bicicleta** e **Usuario**. Ambas devem seguir as convenções de encapsulamento e visibilidade de atributos.

Classe 1: **Bicicleta**

Esta classe representa as bicicletas elétricas espalhadas pela cidade.

- **Atributos (4 atributos protegidos):**
 1. `_codigo` (string): Identificador único da bicicleta.
 2. `_modelo` (string): Modelo da bicicleta (ex: "Urbana", "Mountain").
 3. `_carga_bateria` (float): Percentual da bateria (de 0 a 100).
 4. `_disponivel` (booleano): Indica se a bicicleta está livre para uso.
- **Encapsulamento (Getters e Setters):**
 5. Implemente um **getter** e um **setter** utilizando o decorador `@property` para o atributo `_carga_bateria`. O setter deve validar se a nova carga está rigorosamente entre 0 e 100.
- **Métodos de Operação (3 métodos):**
 1. `usar(minutos)`: Altera o estado da bicicleta. Reduz a bateria proporcionalmente ao tempo de uso (considere que a bateria consome **0.5% por minuto de uso**). Se a carga de bateria chegar a 0, mude o atributo `_disponivel` para `False`.
 2. `recarregar()`: Eleva a bateria para 100 e define `_disponivel` como `True`.
 3. `autonomia_estimada()`: Retorna um valor numérico representando quantos quilômetros a bicicleta ainda pode rodar (considere que cada **1% de bateria equivale a 0.8 km** de autonomia).

Classe 2: **Usuario**

Esta classe representa os clientes cadastrados no sistema.

- **Atributos (4 atributos protegidos):**

4. `_nome` (string): Nome do cliente.
 5. `_saldo` (float): Saldo disponível em Reais para realizar viagens.
 6. `_bike_alugada` (objeto da classe `Bicicleta` ou `None`): Guarda a referência da bicicleta atualmente em uso.
 7. `_viagens_realizadas` (inteiro): Contador de viagens concluídas pelo usuário.
- **Encapsulamento (Getters e Setters):**
 8. Implemente um **getter** e um **setter** usando `@property` para o atributo `_saldo`. O setter deve impedir que depósitos ou alterações de saldo recebam valores negativos (utilize a função `abs ( )` ou lógica condicional para contornar valores negativos).
 - **Métodos de Operação (3 métodos):**
 1. `adicionar_saldo(valor)`: Método simples para incrementar o saldo do usuário através de seu setter.
 2. `alugar_bike(bike)`: Recebe um objeto `Bicicleta`. O aluguel só pode ser efetuado se: o usuário não tiver outra bike alugada, o seu saldo for de no mínimo **R\$ 5,00**, e a bicicleta estiver disponível com carga de bateria superior a **10%**. Se bem-sucedido, associa a bike ao usuário, define a disponibilidade da bike como **False** e imprime uma mensagem de sucesso.
 3. `devolver_bike(tempo_uso)`: Finaliza a viagem. Calcula o custo da corrida (considere a tarifa de **R\$ 0.40 por minuto**). Deduz o custo do saldo do usuário, invoca o método `usar(tempo_uso)` do objeto `Bicicleta` associado para atualizar a bateria, desassocia a bicicleta (voltando o atributo a `None`), disponibiliza a bike novamente para o sistema se ela tiver bateria, e soma 1 ao contador de viagens realizadas.

Critérios de Implementação e Boas Práticas (Guia para os Alunos)

1. **Diagrama de Classes (UML):** Lembre-se de que a representação gráfica de uma classe em UML utiliza um retângulo dividido em três seções: o nome da classe no topo, os atributos (usando o símbolo `-` para privado/protegido) no meio, e os métodos no compartimento inferior.
2. **Inicializador e self:** Utilize o método especial `__init__` para definir como as variáveis serão criadas e use o parâmetro `self` para referenciar o contexto de cada objeto individualmente.
3. **Representação Amigável:** Implemente o método especial `__str__` em ambas as classes para retornar representações amigáveis em formato de texto (string), facilitando a exibição no terminal usando a função `print ( )`.
4. **A filosofia "Consenting Adults":** Embora o Python não bloqueie rigidamente o acesso a dados protegidos como outras linguagens compiladas, use a convenção de **um underline simples** (`_`) para indicar aos outros programadores que o atributo é protegido e não deve ser modificado diretamente fora das classes.

Exemplo de Teste do Aluno (Simulação de Uso)

```
# Criando as instâncias
bike_eletrica = Bicicleta(codigo="BK-100", modelo="Urbana", carga_bateria=90.0,
disponivel=True)
cliente = Usuario(nome="Pedro", saldo=20.00, bike_alugada=None,
viagens_realizadas=0)
```

```
# Demonstração de uso e mensagens entre objetos
print(bike_eletrica)
cliente.alugar_bike(bike_eletrica)
cliente.devolver_bike(minutos_uso=30)

print(cliente)
print(bike_eletrica)
```